

PROJECT SYNOPSIS

CloudRakshak

A Multi-Cloud Cost Guardian for Students, Freelancers & Early-Stage Teams

Real-time cloud spend monitoring, anomaly detection, and WhatsApp/Telegram alerts
for AWS, Azure & GCP accounts with small budgets

Prepared by
Gade Rushikesh

Final Year, B.Tech — Computer Science & Technology
September 2026

CONTENTS

1. Name / Title of the Project.....	1
2. Statement of the Problem.....	2
3. Objective and Scope of the Project.....	2
4. Methodology.....	2
5. Hardware & Software to be Used.....	3
6. Future Work of this Project.....	4
7. Schedule of the Project (Gantt chart / PERT chart).....	4
8. Conclusion.....	5
9. References / Bibliography.....	5

PROJECT SYNOPSIS

CloudRakshak

1. Name / Title of the Project

CloudRakshak — A Multi-Cloud Cost Guardian for Students, Freelancers & Early-Stage Teams.

2. Statement of the Problem

Cloud computing has become the default infrastructure choice even for students, freelancers, and early-stage founders in India, largely due to free-tier programs such as AWS Educate, Azure for Students, and GCP credits. Cloud billing, however, is usage-based and notoriously opaque: a single misconfigured resource, an idle test server left running, or an unexpected data-transfer charge can silently consume an entire month's free credits or push a personal card into an unplanned charge. Cloud providers expose raw billing dashboards such as AWS Cost Explorer, but these are reporting tools, not proactive-protection tools — a user must actively log in to notice overspending, by which time the damage is already done.

Existing third-party cost-optimization tools (Cloudchipr, Vantage, CloudZero, Finout, and India's own Centilytics) are all built for companies with dedicated cloud budgets and DevOps staff, priced from roughly \$49 to \$500+ per month or gated behind a sales call, and none deliver alerts through WhatsApp or Telegram — the channels students and small teams actually check daily. There is currently no free or near-free, self-serve tool that proactively protects a small cloud account from silent overspending.

3. Objective and Scope of the Project

Objectives:

- Connect securely (read-only IAM role) to a user's AWS, Azure, and/or GCP billing account.
- Build a daily cost-ingestion pipeline pulling usage and billing data via each provider's native Cost/Billing API.
- Implement anomaly-detection logic to flag abnormal day-over-day spend increases and identify idle or underutilised resources.
- Deliver real-time, plain-language alerts via WhatsApp and Telegram, alongside a web dashboard.
- Generate a month-end spend forecast scaled and displayed in INR, appropriate for small budgets.
- Support multi-cloud aggregation so a user with resources across two or more providers sees one unified view.
- Validate the system's accuracy against real or realistically simulated student/small-team cloud usage data.

Scope:

The platform covers the full cost-guardianship lifecycle — account onboarding, continuous cost ingestion, anomaly detection, alerting, and forecasting. It supports individual developer and small-team accounts on AWS (primary), with Azure and GCP as stretch integrations, and is accessible via a mobile-responsive web dashboard plus chat-based alerts.

4. Methodology

The project follows an incremental development approach across 5 phases:

Phase	Activity
Phase 1 (Day 1–6)	Requirement study, IAM/security design, database schema design, UI/UX wireframes, environment setup.
Phase 2 (Day 7–18)	Core development — secure AWS IAM read-only integration, Cost Explorer API ingestion pipeline, database storage, and base dashboard using React, Node.js, and PostgreSQL.
Phase 3 (Day 19–28)	Advanced features — anomaly-detection module (moving average/threshold model), idle-resource detection, INR-scaled month-end forecasting, and dashboard visualisations (trend, breakdown, forecast).
Phase 4 (Day 29–35)	Unique features — Telegram Bot API and WhatsApp Cloud API alert integration for real-time, plain-language notifications; optional Azure/GCP multi-cloud integration.
Phase 5 (Day 36–40)	Testing against seeded spend-spike scenarios, bug fixing, performance optimisation, and final deployment/demo preparation.

System Flow:

- User → Sign Up → Create Read-Only IAM Role → Connect Cloud Account(s)
- System → Daily Scheduled Ingestion → Cost Explorer / Billing API → Normalised Storage
- System → Anomaly Detection → Flag Spend Spike / Idle Resource → Generate Alert
- Alert Engine → Telegram Bot API / WhatsApp Cloud API → User Notification
- User → Web Dashboard → View Trend, Breakdown, Forecast, Recommendations

5. Hardware & Software to be Used

Hardware:

Intel Core i3 or above, 8 GB RAM, 20 GB storage, broadband internet, and an active AWS (and optionally Azure/GCP) account for integration testing.

Software:

Category	Technology
Frontend	React, Recharts (for spend/trend visualisation)
Backend	Node.js / Python (serverless functions for scheduled ingestion)
Database	PostgreSQL (Supabase / Neon free tier)

Category	Technology
Cloud Billing APIs	AWS Cost Explorer API (primary); Azure Cost Management API, GCP Billing API (stretch)
Identity & Access	AWS IAM AssumeRole / STS (scoped, read-only, revocable)
Alerting	Telegram Bot API (primary, free); WhatsApp Cloud API (secondary, free tier)
Hosting	Vercel / Netlify (frontend); AWS Lambda (backend functions)
Tools	VS Code, Postman, Git, GitHub

Limitations:

- No automated remediation in this phase — the system alerts and recommends but does not auto-terminate resources, to avoid destructive-action risk during evaluation.
- Anomaly detection uses a lightweight statistical model, not deep learning, due to time and data constraints.
- WhatsApp alerting is limited to the Cloud API's free-tier conversation quota during demonstration.
- Full multi-cloud support (Azure + GCP together with AWS) is a stretch goal dependent on time available.

6. Future Work of this Project

- Automated remediation — one-click or auto-shutdown of confirmed idle/wasteful resources.
- Reserved Instance / Savings Plan purchase recommendations, automated where safe.
- Native Android and iOS companion apps.
- Machine-learning-based anomaly detection to replace the initial threshold model.
- Support for additional providers (DigitalOcean, Oracle Cloud, Cloudflare).
- Team/organisation accounts with role-based access for small multi-person startups.

7. Schedule of the Project

Total Duration: 40 Days

Phase	Activity	Duration
Phase 1	Planning, IAM/security design, database schema	Day 1 – Day 6
Phase 2	Core ingestion pipeline and dashboard development	Day 7 – Day 18
Phase 3	Anomaly detection, idle-resource flagging, forecasting	Day 19 – Day 28
Phase 4	Telegram/WhatsApp alerting and multi-cloud integration	Day 29 – Day 35
Phase 5	Testing, optimisation, deployment and demo prep	Day 36 – Day 40

8. Conclusion

CloudRakshak is a full-stack, cloud-native application that addresses a real, evidence-based gap in the cloud cost-management market — every existing tool, global or Indian, is priced and designed for companies with dedicated cloud budgets, leaving students, freelancers, and early-stage teams structurally unprotected from silent overspending. By combining read-only multi-cloud billing integration, lightweight anomaly detection, INR-scaled forecasting, and WhatsApp/Telegram-native alerting, the project delivers genuine, everyday value to a large and underserved segment of Indian cloud users, while demonstrating practical full-stack, cloud API, and DevOps skills valuable for placement.

9. References / Bibliography

- [1] AWS Cost Explorer API Documentation. Available: <https://docs.aws.amazon.com/cost-management/>
- [2] AWS IAM — AssumeRole / STS Documentation. Available: <https://docs.aws.amazon.com/IAM/>
- [3] Telegram Bot API Documentation. Available: <https://core.telegram.org/bots/api>
- [4] WhatsApp Cloud API Documentation, Meta for Developers. Available: <https://developers.facebook.com/docs/whatsapp/cloud-api>
- [5] PostgreSQL Documentation. Available: <https://www.postgresql.org/docs/>
- [6] React Documentation. Available: <https://react.dev/>
- [7] Node.js Documentation. Available: <https://nodejs.org/en/docs>
- [8] MarketsandMarkets, "Cloud FinOps Market Report 2025–2030."
- [9] Cloudchipr, "Best Cloud Cost Optimization Tools for 2026."
- [10] Centilytics Company Profile — Tracxn, GetLatka, FinOps Foundation Vendor Listing.